
DriveMart

A Cloud-Based Full-Stack Marketplace for
Buying and Selling Pre-Owned Vehicles

Comprehensive Technical Project Report

Project Type Full-Stack Web Application

Architecture Client-Server (REST API)

Backend Stack Node.js, Express, TypeScript, Prisma

Database PostgreSQL 16

Frontend Stack React, TypeScript, Vite

Deployment Docker + Render Cloud

Live Demo: <https://drivemart-fullstack-1.onrender.com>

Source Code: <https://github.com/vaishnavi170117-ops/DriveMart-FullStack>

July 20, 2026

Contents

1	Executive Summary	3
2	Introduction and Objectives	3
2.1	Background	3
2.2	Problem Statement	4
2.3	Project Objectives	4
2.4	Scope and Deliverables	4
2.5	Intended Audience	4
3	Technology Stack	4
3.1	Rationale for Key Choices	5
3.2	Design Philosophy: Graceful Degradation	6
4	System Architecture	6
4.1	Architectural Principles	6
4.2	Backend Layered Structure	7
4.3	Request Lifecycle	7
5	Repository Structure	8
6	Core Features and Functional Modules	8
6.1	Authentication Module	8
6.2	Vehicle Listing Module	9
6.3	Payments Module	9
7	Security Design	9
7.1	Authentication and Session Security	9
7.2	Data and Transport Protection	10
8	Database Design	10
8.1	Entity Relationships	10
8.2	Prisma Tooling	10
9	API Design and Documentation	10
9.1	API Conventions	11
10	CI/CD and Deployment Pipeline	11
10.1	Infrastructure as Code	11
10.2	Deployed Services	12
10.3	Environment Configuration	12
10.4	Local Development Workflow	12
11	Testing and Quality Assurance	13
11.1	Testing Strategy	13
11.2	Code Quality Enforcement	13
11.3	Available Developer Scripts	13
12	Project Status and Roadmap	13
12.1	Frontend Development Phases	14
12.2	Future Enhancements	14

13 Challenges and Engineering Highlights	15
13.1 Challenges Addressed	15
13.2 Engineering Highlights	15
14 Conclusion	15
A Appendix A: Technology Reference	16
B Appendix B: Quick-Reference Endpoints	16

1 Executive Summary

DriveMart is a cloud-based, full-stack marketplace platform designed to facilitate the buying and selling of pre-owned vehicles. It provides an end-to-end digital experience in which sellers can list vehicles and prospective buyers can browse, search, and transact securely. The system is engineered with a clear separation between a robust backend REST API and a modern single-page frontend application, an approach that allows each tier to evolve, scale, and be tested independently.

The backend is a production-grade API built with Node.js, Express, and TypeScript, backed by a PostgreSQL relational database accessed through the Prisma ORM. It is fully containerized with Docker and deployed to the Render cloud platform through an infrastructure-as-code blueprint. The frontend is a responsive web application built with React, TypeScript, and the Vite build tool, communicating with the backend exclusively over HTTPS using JSON payloads.

The project emphasizes engineering rigor. The backend is feature-complete and has been validated with an automated test suite of **408 passing tests** executed against a live database, providing a high degree of confidence in its correctness and stability. The platform integrates several third-party services, including Cloudinary for image storage, Razorpay for payments, SMTP for transactional email, and Google OAuth for federated authentication. Crucially, every one of these integrations is optional: the application is designed to degrade gracefully and remain operational even when their credentials are absent.

This report documents the project's objectives, technology choices, system architecture, data model, API surface, security posture, deployment pipeline, testing strategy, and current status. It is intended to serve both as a technical reference for developers and as an evaluative overview for reviewers and stakeholders.

Table 1: Project At-a-Glance

Attribute	Value
Domain	E-commerce / Automotive Marketplace
Primary Language	TypeScript (98.7% of codebase)
Architecture Pattern	Layered REST API + SPA client
Backend Test Coverage	408 automated passing tests
Backend Status	Feature-complete, verified against live database
Frontend Status	In progress (Phases 1-3 complete)
Containerization	Docker (multi-stage build)
Cloud Host	Render (web service + managed PostgreSQL)
API Documentation	Swagger / OpenAPI (<code>/api-docs</code>)
Repository Model	Monorepo (Backend + Frontend)

2 Introduction and Objectives

2.1 Background

The market for pre-owned vehicles is large but historically inefficient. Transactions frequently occur through classified listings, social-media groups, and word-of-mouth, none of which provide a structured, trustworthy, or secure environment. Buyers struggle to compare vehicles on consistent attributes, while sellers lack a reliable channel to reach a wide audience and receive secure payment. These inefficiencies motivate a purpose-built digital marketplace.

2.2 Problem Statement

The pre-owned vehicle market is fragmented, with buyers and sellers relying on informal channels that lack transparency, secure payment mechanisms, and structured listings. There is a clear need for a unified platform that standardizes vehicle information, authenticates its users, safeguards payments, and scales reliably in the cloud. DriveMart is conceived to meet exactly this need.

2.3 Project Objectives

The project is guided by the following concrete objectives:

- Provide a secure authentication system supporting both password-based and federated (Google OAuth) login, with token-based session management.
- Enable sellers to create, manage, and publish detailed vehicle listings, including image uploads to a cloud media store.
- Allow buyers to browse, search, and filter available vehicles, and to complete purchases through an integrated payment gateway.
- Deliver a maintainable, well-tested, and cloud-deployable codebase that follows modern engineering practices such as type safety, automated testing, and infrastructure as code.
- Ship comprehensive, interactive API documentation to support ongoing frontend development and future third-party integration.
- Ensure operational resilience so that the platform can be deployed incrementally, functioning even before every optional integration is configured.

2.4 Scope and Deliverables

The current scope covers a complete, feature-complete backend API and a frontend web client under active phased development. The backend supports authentication, vehicle listing management, media handling, payments, and email notifications. The database schema is deliberately frozen at version 1.0 to ensure structural stability. Deliverables include the backend service, the deployment blueprint, the Swagger API specification, a deployment guide, and the frontend application shell with authentication and layout implemented.

2.5 Intended Audience

This document targets three audiences: developers who will extend or maintain the codebase and need architectural context; reviewers and evaluators who require a clear account of scope, quality, and status; and stakeholders who wish to understand the platform's capabilities and readiness at a high level.

3 Technology Stack

The technology choices reflect a modern, type-safe, and cloud-native philosophy. The codebase is predominantly written in TypeScript, ensuring a single, statically typed language governs both client and server logic. This reduces a broad class of runtime errors, improves editor tooling and refactoring safety, and lowers the cognitive cost of moving between the two halves of the stack.

Table 2: Complete Technology Stack

Layer	Technology	Purpose
Backend	Node.js 18+	JavaScript runtime environment
	Express	Web framework and routing
	TypeScript	Type-safe application language
	Prisma ORM	Database access and migrations
Database	PostgreSQL 16	Relational data persistence
Frontend	React	Component-based UI library
	TypeScript	Type-safe frontend logic
	Vite	Build tool and dev server
DevOps	Docker + Compose	Containerization and orchestration
	Render	Cloud hosting (blueprint deployment)
Docs & Quality	Swagger / OpenAPI	Interactive API documentation
	ESLint + Prettier	Linting and code formatting
Integrations	Cloudinary	Image upload and storage (optional)
	Razorpay	Payment gateway (optional)
	SMTP	Transactional email delivery (optional)
	Google OAuth	Federated sign-in (optional)

3.1 Rationale for Key Choices

Each major technology was selected for a specific reason rather than by default:

- **TypeScript everywhere** unifies the language across client and server, enabling shared mental models and reducing integration errors at the boundary between them.
- **Express** is a minimal, well-understood, and highly extensible web framework, giving fine-grained control over middleware, routing, and error handling.
- **Prisma ORM** provides a type-safe query layer generated directly from the schema, along with a first-class migration system and a visual database browser, greatly reducing the risk of malformed queries.
- **PostgreSQL** is a mature, standards-compliant relational database well suited to the transactional, relational nature of a marketplace (users, listings, and purchases with clear relationships).
- **React with Vite** offers a fast, component-driven frontend with near-instant hot-module reloading during development and optimized production builds.
- **Docker and Render** together make the deployment reproducible and portable, decoupling the application from any single machine's configuration.

3.2 Design Philosophy: Graceful Degradation

A defining engineering decision is that all third-party integrations are optional and independently toggleable. The application degrades gracefully when credentials are unset: without SMTP there is no email delivery, without Cloudinary there is no image upload, without Razorpay the checkout is simulated, and without Google credentials the “Sign in with Google” option is simply hidden. This design yields two practical benefits. First, it allows incremental deployment, since the platform can boot and serve traffic before every external account is provisioned. Second, it improves resilience, because the failure or absence of a non-critical external service does not prevent the core application from starting.

4 System Architecture

DriveMart follows a classic three-tier client-server architecture. The React single-page application forms the presentation tier, the Express REST API and its business logic form the application tier, and PostgreSQL forms the data tier. The Prisma ORM sits between the application and data tiers as a type-safe access layer. External services are invoked from the application tier only when their respective integrations are configured.

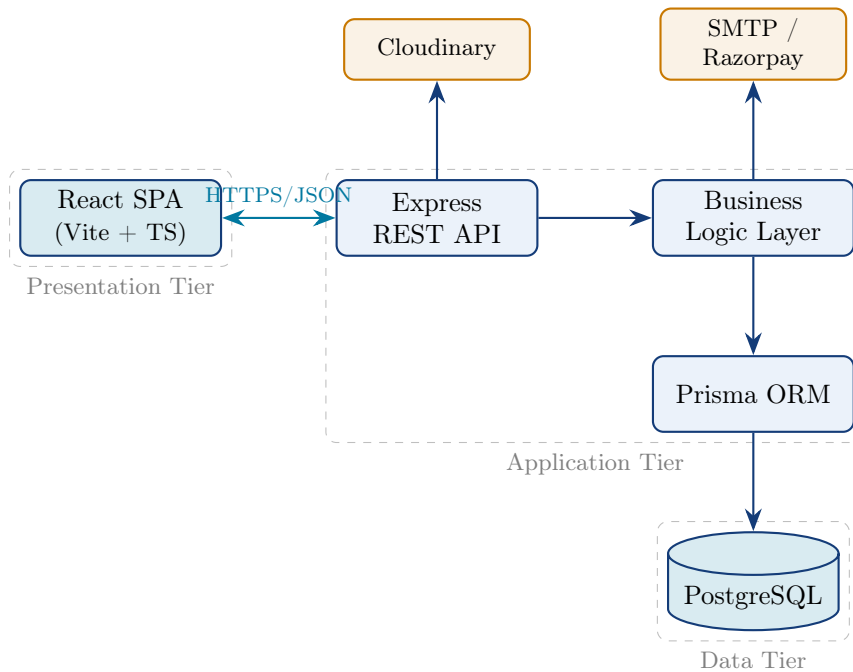


Figure 1: DriveMart three-tier system architecture with external service integrations.

4.1 Architectural Principles

The architecture is guided by a small set of principles. *Separation of concerns* ensures that presentation, business logic, and persistence are distinct and independently replaceable. *Statelessness* at the API layer means each request carries its own authentication context (a JWT), allowing horizontal scaling without server-side session affinity. *Configuration through environment* keeps secrets and environment-specific values out of the codebase, following twelve-factor principles. *Fail-safe defaults* ensure that missing optional configuration disables a feature rather than crashing the service.

4.2 Backend Layered Structure

Within the application tier, the backend is organized into clearly delineated layers, each with a single responsibility. This layering makes the code easier to test, reason about, and extend.

Table 3: Backend Application Layers

Layer	Responsibility
Configuration	Environment loading, application settings, and feature toggles derived from environment variables
Routing	Express route definitions that map HTTP methods and URLs to controller handlers
Controllers	Request handling, input validation, orchestration, and response shaping
Business Logic	Core domain rules governing authentication, listings, and payments
Data Access	The Prisma client, providing type-safe queries and transactions against PostgreSQL
Middleware	Cross-cutting concerns: authentication, error handling, request logging, and CORS
Documentation	The Swagger / OpenAPI specification and its interactive UI

4.3 Request Lifecycle

A typical authenticated request traverses the stack in a predictable sequence. Understanding this flow clarifies where validation, security, and error handling occur.

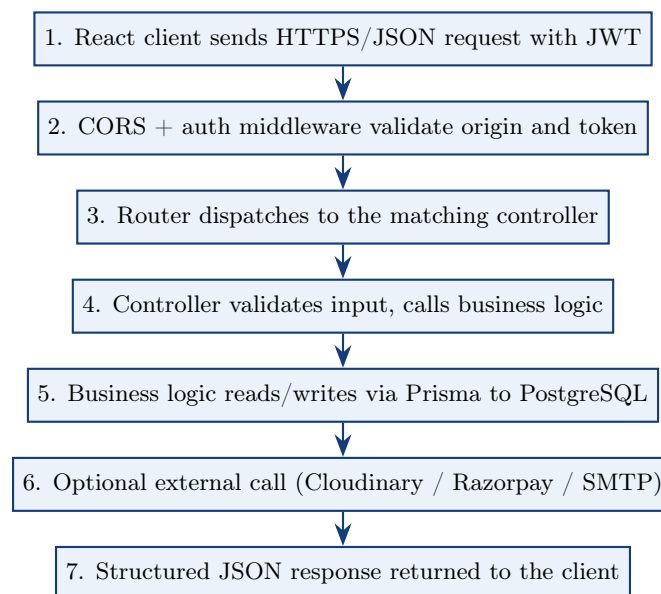


Figure 2: End-to-end lifecycle of an authenticated API request.

External calls in step six are conditional: they execute only when the corresponding integration is configured, consistent with the graceful-degradation philosophy. Errors at any stage are captured by a centralized error-handling middleware, which produces a consistent JSON error shape rather than leaking stack traces.

5 Repository Structure

The project is organized as a monorepo, keeping the backend and frontend in a single version-controlled repository. This simplifies coordinated changes, shared documentation, and a unified history, while still allowing each half to be built and deployed independently.

```
DriveMart/
|-- Backend/           Node.js + Express + TypeScript + Prisma API
|   '-- server/       Application source, Dockerfile, Prisma schema
|-- Frontend/         React + TypeScript + Vite web application
|-- render.yaml       Render cloud deployment blueprint
|-- DEPLOYMENT.md     Deployment guide
|-- README.md         Project overview
'-- .gitignore
```

Listing 1: Project directory structure

The `Backend/server` directory houses the application source, its Dockerfile, and the frozen Prisma schema. The `Frontend` directory contains the React application and its own environment template. Top-level files provide project-wide concerns: `render.yaml` declares the cloud infrastructure, `DEPLOYMENT.md` documents the deployment procedure, and `README.md` orients new contributors.

6 Core Features and Functional Modules

The platform’s functionality is organized into cohesive modules, each responsible for a distinct area of the marketplace experience.

Table 4: Functional Modules

Module	Description
Authentication	User registration, password-based login, JWT access and refresh tokens, a password-reset flow, and optional Google OAuth login.
Vehicle Listings	Creation, retrieval, updating, and deletion of pre-owned vehicle listings with rich, structured attributes.
Media Management	Vehicle image upload and storage via the Cloudinary integration, with graceful fallback when unconfigured.
Search & Browse	Buyer-facing browsing, searching, and filtering across the catalogue of available vehicles.
Payments	Secure checkout and transaction processing through Razorpay, including server-side webhook verification.
Notifications	Transactional email delivery for events such as verification and password reset via SMTP.
Health Monitoring	A <code>GET /api/health</code> endpoint returning service uptime, used by container and load-balancer health checks.

6.1 Authentication Module

Authentication is the gateway to the platform. It supports traditional email-and-password credentials as well as federated login through Google OAuth. Upon successful login, the server

issues a short-lived access token and a longer-lived refresh token, both signed with independent secrets. The refresh token enables seamless session renewal without forcing the user to re-enter credentials, while the short lifetime of the access token limits the window of misuse if it is ever compromised. A dedicated password-reset flow issues time-limited reset links, delivered by email when SMTP is configured.

6.2 Vehicle Listing Module

The listing module provides full lifecycle management of vehicle records. Sellers can create listings with structured attributes, update them as circumstances change, and remove them when a sale completes. Each listing can reference images stored in Cloudinary. The buyer-facing side of this module exposes browsing and search capabilities so that prospective purchasers can locate vehicles matching their criteria.

6.3 Payments Module

Payments are handled through Razorpay. The server initiates a checkout, and payment confirmation is verified server-side through webhooks, ensuring that the platform's record of a transaction is authoritative rather than relying solely on client-reported success. When Razorpay credentials are not configured, checkout operates in a simulated mode, allowing the rest of the application to be exercised without a live payment account.

7 Security Design

Security is treated as a first-class concern throughout the platform rather than an afterthought.

Table 5: Security Measures

Measure	Implementation
Token-based auth	JWT access and refresh tokens signed with separate secrets, each at least 32 characters long.
Origin control	CORS strictly configured to accept requests only from the designated frontend origin.
Secret management	Secrets injected as environment variables at deploy time and never committed to source control.
Payment integrity	Razorpay transactions verified server-side via webhook signatures.
Transport security	All client-server communication conducted over HTTPS.
Least exposure	Centralized error handling avoids leaking stack traces or internal details to clients.

7.1 Authentication and Session Security

The separation of access and refresh secrets means that compromising one token type does not automatically compromise the other. Requiring secrets of at least 32 characters raises the bar against brute-force attacks on token signatures. Because the API is stateless with respect to sessions, revocation and renewal are handled through the token lifecycle rather than server-side session storage, which also aids horizontal scalability.

7.2 Data and Transport Protection

Cross-Origin Resource Sharing is deliberately restrictive: the backend rejects requests from any origin other than the configured frontend, mitigating a class of cross-site attacks. Sensitive configuration, including JWT secrets and payment keys, is supplied exclusively through environment variables at deploy time, keeping it out of the repository and version history. All traffic between the client and server travels over HTTPS.

8 Database Design

The persistence layer uses PostgreSQL 16, accessed exclusively through the Prisma ORM. The schema is version-controlled and **frozen at v1.0**, located at `src/prisma/schema.prisma`, and must not be structurally modified without an explicit architecture review. This freeze policy is a deliberate stability guarantee: it prevents accidental structural drift and ensures that all code is written against a known, stable data model.

Table 6: Core Domain Entities

Entity	Description
User	Registered accounts, including credentials, profile data, and the authentication provider used.
Vehicle	Pre-owned vehicle listings with make, model, specifications, price, and associated images.
Transaction	Purchase records linking buyers, listings, and their payment status.
Session / Token	Refresh-token records that underpin secure, renewable session management.

8.1 Entity Relationships

The domain model is relational by nature. A user may act as a seller, owning many vehicle listings, and as a buyer, participating in many transactions. Each vehicle listing belongs to exactly one selling user, and each transaction links a buyer, a vehicle, and a resulting payment status. Refresh-token records are associated with the user to whom they were issued. These relationships map naturally onto a relational schema with foreign-key constraints, which PostgreSQL enforces to preserve referential integrity.

8.2 Prisma Tooling

Prisma contributes far more than query execution. It provides type-safe queries generated directly from the schema, an automated migration system (`prisma migrate`) that versions structural changes, a client generator (`prisma generate`) that keeps application types in sync with the database, a visual browser (`prisma studio`) for inspecting data during development, and a seeding mechanism (`prisma seed`) for populating reference data. Together these tools make database work safer and more productive.

9 API Design and Documentation

The API is exposed under a configurable base path, defaulting to `/api`, and follows RESTful conventions in which resources are addressed by URL and manipulated through standard HTTP

methods. The complete surface is documented using the OpenAPI (Swagger) specification and served interactively at `/api-docs`, allowing developers to explore endpoints and issue trial requests directly from the browser.

Table 7: Representative API Surface

Method	Endpoint	Purpose
GET	<code>/api/health</code>	Service health and uptime check
POST	<code>/api/auth/register</code>	Create a new user account
POST	<code>/api/auth/login</code>	Authenticate and issue tokens
POST	<code>/api/auth/refresh</code>	Refresh an access token
GET	<code>/api/vehicles</code>	List / search vehicles
POST	<code>/api/vehicles</code>	Create a vehicle listing
GET	<code>/api/vehicles/:id</code>	Retrieve a single listing
POST	<code>/api/payments</code>	Initiate a Razorpay checkout

Note: endpoint names are representative of the documented API surface exposed via Swagger.

9.1 API Conventions

The API adopts consistent conventions to make it predictable. Resource collections are addressed by plural nouns, individual resources by an identifier segment, and actions are expressed through HTTP verbs rather than encoded in the URL. Responses use standard HTTP status codes and a uniform JSON envelope, and errors share a single, documented error shape. This consistency reduces the learning curve for frontend and third-party developers and makes client code simpler to write and maintain.

10 CI/CD and Deployment Pipeline

DriveMart uses a container-based deployment workflow orchestrated through a Render Blueprint declared in `render.yaml`. The backend is built from a multi-stage Dockerfile that defines separate *development* (hot-reload) and *production* targets; Render's Docker builder targets the production stage, producing a lean image that runs the compiled application.

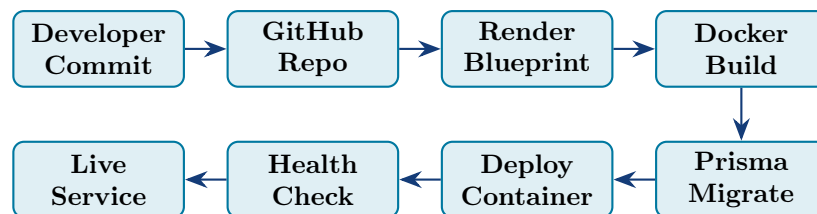


Figure 3: Continuous deployment pipeline from commit to live service on Render.

10.1 Infrastructure as Code

Declaring the infrastructure in `render.yaml` means the entire backend service and its managed PostgreSQL database can be provisioned together from a single blueprint import. This makes environments reproducible and reviewable: the infrastructure is versioned alongside the application code, and changes to it pass through the same history and review process as any other

change. Secrets such as the JWT signing keys are intentionally excluded from the blueprint and are supplied interactively during setup.

10.2 Deployed Services

Table 8: Render Deployed Resources

Resource	Type	Configuration
drivemart-backend	Web (Docker)	Production target, health check at <code>/api/health</code> , port 5000
drivemart-postgres	Database	Managed PostgreSQL, database <code>drivemart_db</code>

10.3 Environment Configuration

Configuration is fully externalized through environment variables, following twelve-factor principles. Required variables must be present for the service to operate, while optional variables enable individual integrations when supplied.

Table 9: Key Environment Variables

Variable	Required	Purpose
<code>NODE_ENV</code>	Yes	Runtime environment (production)
<code>PORT</code>	Yes	Service port (5000)
<code>API_BASE_PATH</code>	Yes	API route prefix (<code>/api</code>)
<code>DATABASE_URL</code>	Yes	PostgreSQL connection string
<code>JWT_ACCESS_SECRET</code>	Yes	Access-token signing key (≥ 32 chars)
<code>JWT_REFRESH_SECRET</code>	Yes	Refresh-token signing key (≥ 32 chars)
<code>CORS_ORIGIN</code>	Yes	Allowed frontend origin
<code>CLOUDINARY_*</code>	Optional	Image storage credentials
<code>RAZORPAY_*</code>	Optional	Payment gateway credentials
<code>SMTP_*</code>	Optional	Email delivery credentials
<code>GOOGLE_CLIENT_*</code>	Optional	Google OAuth credentials

10.4 Local Development Workflow

For local development, Docker Compose orchestrates the database and the backend together with hot reload, closely mirroring the production topology while remaining convenient to run on a single machine.

```
cd Backend
docker compose up --build
# API:      http://localhost:5000/api
# API docs: http://localhost:5000/api-docs
```

Listing 2: Backend local startup

The frontend runs as a separate Vite development server, configured through its own environment file.

```
cd Frontend
cp .env.example .env
npm install
npm run dev
# App: http://localhost:5173
```

Listing 3: Frontend local startup

11 Testing and Quality Assurance

Quality assurance is a first-class concern. The backend ships with a comprehensive automated test suite whose results are a key indicator of the project’s maturity.

Table 10: Quality Assurance Summary

Aspect	Detail
Automated Tests	408 passing tests
Verification	Executed against a live database
Backend Status	Feature-complete across all modules
Linting	ESLint enforced (<code>npm run lint</code>)
Formatting	Prettier enforced (<code>npm run format</code>)
Type Safety	TypeScript across the full stack

11.1 Testing Strategy

The suite of 408 tests is run against a live database rather than only against mocks, which means the tests exercise real query behaviour, constraints, and data integrity rather than an idealized stand-in. This gives strong assurance that the backend behaves correctly under realistic conditions. Combined with full-stack static typing, the result is a defense-in-depth approach to correctness: the type system catches whole categories of errors at compile time, while the test suite validates runtime behaviour and integration with the database.

11.2 Code Quality Enforcement

Beyond functional testing, the project enforces consistent code quality through automated tooling. ESLint applies a shared set of lint rules to catch suspect patterns, and Prettier enforces a uniform code style so that formatting is never a subject of debate or a source of noisy diffs. These tools run as ordinary npm scripts and can be integrated into a continuous-integration gate.

11.3 Available Developer Scripts

12 Project Status and Roadmap

The project has reached a state where the backend is complete and deployed, and the frontend is progressing through a disciplined, phased plan.

Table 11: NPM Scripts

Command	Purpose
<code>npm run dev</code>	Start dev server with hot reload (tsx watch)
<code>npm run build</code>	Compile TypeScript to <code>dist/</code>
<code>npm start</code>	Run the compiled production build
<code>npm run prisma:generate</code>	Regenerate the Prisma client
<code>npm run prisma:migrate</code>	Create / apply a development migration
<code>npm run prisma:studio</code>	Open Prisma Studio database browser
<code>npm run prisma:seed</code>	Run the database seed script
<code>npm run lint / lint:fix</code>	Lint the codebase
<code>npm run format</code>	Format code with Prettier
<code>npm test</code>	Run the automated test suite

Table 12: Component Status

Component	Status	Details
Backend	Complete	All modules implemented, 408 tests passing, verified on live DB
Frontend	In Progress	Phases 1–3 complete: setup, authentication, layout
Deployment	Live	Backend and database deployed on Render
Documentation	Complete	Swagger API docs and deployment guide

12.1 Frontend Development Phases

The frontend follows an incremental plan in which each phase delivers a coherent, testable increment of functionality.

1. **Phase 1 — Project Setup:** Vite + React + TypeScript scaffolding and tooling. (Complete)
2. **Phase 2 — Authentication:** Login, registration, and session management UI. (Complete)
3. **Phase 3 — Application Layout:** Core navigation and layout shell. (Complete)
4. **Phase 4+ — Feature Screens:** Vehicle listing, browsing, and checkout screens. (Planned)

12.2 Future Enhancements

Several enhancements are natural extensions of the current foundation:

- Completion of frontend feature screens and dedicated buyer and seller dashboards.
- Automated CI test gating via GitHub Actions so that tests must pass before any deployment.
- Advanced search with faceted filtering and multi-attribute sorting.
- A rating and review system to build trust between buyers and sellers.
- In-app messaging so buyers and sellers can communicate without leaving the platform.

- Observability improvements such as structured logging aggregation and performance monitoring.

13 Challenges and Engineering Highlights

13.1 Challenges Addressed

Building a marketplace of this kind surfaces several recurring challenges. Coordinating multiple optional external services without allowing any of them to become a single point of failure required the graceful-degradation design. Guaranteeing data-model stability across a growing codebase motivated the decision to freeze the schema at v1.0. Achieving confidence in correctness demanded testing against a live database rather than mocks alone. Each of these challenges was met with a deliberate architectural response rather than an ad-hoc workaround.

13.2 Engineering Highlights

The following table distills the qualities that most distinguish the implementation.

Table 13: Engineering Highlights

Highlight	Significance
End-to-end type safety	TypeScript spans both client and server, reducing runtime errors and improving maintainability.
Comprehensive testing	408 automated tests verified against a live database provide strong confidence in backend correctness.
Reproducible deployment	A single Render Blueprint provisions the backend and managed PostgreSQL together, enabling one-click setup.
Multi-stage Docker build	Separate development and production targets keep the production image lean while preserving hot reload locally.
Frozen schema (v1.0)	A version-locked Prisma schema guarantees data-model stability and prevents accidental structural drift.
Optional integrations	Graceful degradation lets the platform boot and run even before external service credentials are supplied.
Self-documenting API	Swagger/OpenAPI at <code>/api-docs</code> allows interactive exploration and eases future integration.

14 Conclusion

DriveMart demonstrates a well-architected, cloud-native full-stack application built on modern, type-safe technologies. The backend is production-ready, feature-complete, and thoroughly tested, while the frontend is progressing through a disciplined phased development plan. The use of Docker containerization, a Render Blueprint for reproducible deployment, comprehensive Swagger documentation, a rigorous test suite executed against a live database, and a graceful-degradation strategy for optional integrations collectively reflect sound software engineering practice. Taken together, these qualities give the project a solid, extensible foundation on which a scalable pre-owned vehicle marketplace can continue to be built.

A Appendix A: Technology Reference

Table 14: Technology Versions and Roles

Technology	Version	Role
Node.js	18+	Server runtime
PostgreSQL	16	Relational database
TypeScript	Latest	Primary language (98.7% of code)
Prisma	v1.0 schema	ORM and migrations
React	Current	Frontend UI library
Vite	Current	Frontend build tool
Docker	Multi-stage	Containerization
Render	Blueprint	Cloud hosting

B Appendix B: Quick-Reference Endpoints

For convenience, the primary service URLs are collected here.

Table 15: Service URLs

Resource	URL / Path
Live Demo	https://drivemart-fullstack-1.onrender.com
Source Repository	https://github.com/vaishnavi170117-ops/DriveMart-FullStack
API Base Path	/api
Health Check	/api/health
Interactive API Docs	/api-docs
Local Backend	http://localhost:5000/api
Local Frontend	http://localhost:5173

End of Report